

RunIQ

Personal Running Intelligence Platform

Project Type: End-to-End ML / AI Portfolio Project

Target User: Single-user — personal use

Development Scope: Strong Portfolio Project

Estimated Duration: 10–14 weeks

Primary Data Sources: Garmin / Strava

Core External Data: Weather + Air Quality

Advanced Module: Running Video / Pose Analysis

1. Project Vision

1.1 Product Idea

RunIQ is a personalized running intelligence platform that combines:

- Historical running data
- Garmin / Strava activity data
- Heart rate
- Pace
- Cadence
- Elevation
- Training volume
- Weather conditions
- Air quality
- Race goals
- Machine learning
- RAG / LLM-based analysis

to help a runner understand:

How am I performing, why is my performance changing, and what should I do next?

The platform is not intended to replace Strava or Garmin as an activity tracker.

Instead, it focuses on **personalized intelligence and decision support**.

2. Product Philosophy

The core philosophy is:

You vs. Yourself

Instead of comparing the user with professional athletes or generic population averages, RunIQ learns the user's own baseline.

For example:

Instead of:

"30°C is hot."

RunIQ should eventually be able to say:

"Your running performance tends to decline when temperature exceeds approximately 29°C, especially when humidity is above 75%."

Instead of:

"Your pace was slower today."

RunIQ should say:

"Today's pace was approximately 8 sec/km slower than your historical baseline under similar distance, training and environmental conditions."

3. Core Intelligence Loop

The overall system follows:

```
Collect
↓
Validate
↓
Understand
↓
Analyze
↓
Predict
```

↓
Explain
↓
Recommend
↓
Run Again
↓
Collect New Data

The long-term goal is to create a feedback loop where the system becomes increasingly personalized as more running data is collected.

4. Target User

Primary User

A recreational / amateur runner who:

- Runs regularly
- Uses Garmin / Strava
- Has accumulated historical training data
- Has a race or performance goal
- Wants to understand performance rather than simply record activities
- Is interested in data-driven training
- Wants personalized insights

Initial Product Scope

The platform is intentionally designed as a **single-user system**.

There is no need to build:

- User registration
- Multi-user authorization
- Subscription system
- Social features
- Public profiles

during the initial project.

However, the architecture should avoid making future multi-user expansion impossible.

5. Data Sources

5.1 Running Data

Primary sources:

- Garmin
- Strava

Current data availability:

- Approximately 150 runs in the current year
- Heart rate
- Cadence
- Elevation
- Elevation gain
- Distance
- Duration
- Pace

Potential future data:

- GPS route
- Running power
- Ground contact time
- Vertical oscillation
- Stride length
- Training effect
- Recovery information
- Sleep / HRV if available

6. External Environmental Data

6.1 Weather

Primary source:

OpenWeather One Call API 4.0

Potential variables:

- Temperature
- Feels-like temperature
- Humidity

- Wind speed
- Wind direction
- Precipitation
- Rain probability
- Cloud cover
- Weather condition

The platform should associate environmental conditions with each running activity.

Example:

Run #143

Distance	8.2 km
Pace	5:47/km
Avg HR	152 bpm
Temperature	30.2°C
Feels Like	34.7°C
Humidity	78%
Wind	11 km/h
Rain	0.8 mm

6.2 Air Quality

Primary source:

Google Maps Air Quality API

Potential variables:

- AQI
- PM2.5
- PM10
- Ozone
- Other available pollutants

The system should store the environmental snapshot associated with each activity rather than repeatedly requesting historical data unnecessarily.

7. Main Product Modules

RunIQ consists of six major modules:

1. Data Platform
2. Running Analytics
3. ML Intelligence
4. Goal & Training Intelligence
5. AI Running Coach
6. Running Form Analysis

The first five form the main project.

Running Form Analysis is an advanced extension.

8. Module 1 — Data Platform

Objective

Build a reliable pipeline that converts raw Garmin / Strava data into clean, structured, analysis-ready data.

Features

Data Import

Support:

- CSV
- GPX
- FIT if practical
- Strava export
- Future Strava API integration

Data Validation

Validate:

- Missing values
- Invalid timestamps
- Duplicate activities
- Invalid distance
- Invalid duration

- Unrealistic HR
- Unrealistic cadence
- Missing location information

Data Normalization

Standardize:

- Units
- Timestamps
- Pace
- Distance
- Heart rate
- Elevation
- Activity type

Database

Store normalized data in PostgreSQL.

9. Suggested Data Model

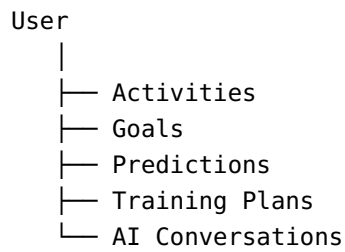
Core entities:

```
activities
activity_metrics
weather_conditions
air_quality
training_load
predictions
goals
insights
```

Conceptually:

```
Activity
|
├─ Metrics
├─ Weather
├─ Air Quality
└─ Derived Features
```

Later:



10. Module 2 — Running Analytics

The dashboard should not simply reproduce Strava.

The purpose is to answer:

What is happening to my running?

10.1 Basic Statistics

Display:

- Total distance
- Number of runs
- Total duration
- Average pace
- Average HR
- Average cadence
- Elevation gain
- Weekly mileage
- Monthly mileage
- Longest run
- Fastest run

11. Performance Analytics

Track:

- Pace trends
- HR trends
- Cadence trends
- Distance progression
- Long-run progression

- Easy-run performance
- Workout performance
- Race performance

Potential charts:

Pace vs. Time

HR vs. Pace

Weekly Mileage

Cadence Trend

Elevation vs. Performance

12. Personal Baseline

One of the most important features.

The system calculates the user's typical:

- Easy-run pace
- Typical HR
- Typical cadence
- Weekly mileage
- Long-run distance
- Workout intensity
- Performance range

Example:

YOUR BASELINE

Easy Pace	5:35/km
Typical HR	148 bpm
Cadence	171 spm
Weekly Volume	31 km
Long Run	11 km

The baseline becomes the reference point for later intelligence.

13. Similar Run Engine

For each activity, find historically similar runs based on:

- Distance
- Workout type
- Elevation
- Environmental conditions
- Training context

Example:

Today's Run

8.2 km
5:47/km
152 bpm

Similar Historical Runs

8.0 km	5:39/km	150 bpm
8.4 km	5:44/km	153 bpm
7.9 km	5:42/km	151 bpm

Then generate:

You were approximately 7 sec/km slower than your historical baseline under similar conditions.

This is one of the key personalized features of RunIQ.

14. Weather Intelligence

Objective

Understand how environmental conditions affect the individual runner.

Variables:

- Temperature
- Humidity
- Feels-like temperature
- Wind

- Rain
- Air quality

15. Personal Weather Profile

The system learns relationships such as:

Temperature

20–24°C	Strong performance
24–28°C	Normal
28–32°C	Moderate slowdown
32°C+	Significant slowdown

And:

Humidity

<60%	Normal
60–75%	Slight effect
75–85%	Moderate effect
85%+	Larger effect

These ranges must be derived from the user's data rather than hard-coded as universal truths.

16. Weather-Adjusted Performance

Potential feature:

Actual Pace

5:48/km

Estimated Weather-adjusted Pace

5:32/km

Environmental Effect

+16 sec/km

The language should remain scientifically careful.

Use:

"The model estimates..."

rather than:

"Weather caused..."

The model should attempt to control for:

- Fitness
- Distance
- Training load
- Elevation
- Heart rate
- Workout type

before attributing performance differences to environmental factors.

17. Air Quality Intelligence

Analyze relationships between:

- AQI
- PM2.5
- Other pollutants
- Pace
- HR
- Running duration

Potential insight:

Your recent runs under higher AQI conditions show higher heart rate at comparable pace.

Again, this should be presented as an observed/modelled relationship rather than medical causation.

18. Module 3 — Training Intelligence

Training Load

Calculate training load using appropriate metrics based on:

- Duration
- Distance
- Intensity
- HR
- Workout type

Track:

7-day load
28-day load
Training frequency
Weekly volume
Long-run volume
Intensity distribution

19. Fitness vs. Fatigue

A key intelligence feature.

The system attempts to distinguish:

Fitness ↑
Fatigue ↑↑

from:

Fitness ↓
Fatigue ↓

This helps explain situations where recent performance temporarily decreases despite increasing training volume.

The system should present these as model-derived indicators, not medical measurements.

20. Training Timeline

Automatically visualize the user's training history.

Example:

```
January
  Base Building

February
  Base Building

March
  Speed Development

April
  Race Preparation

May
  Recovery
```

Initially this can be rule-based.

Later, clustering or change-point detection can be explored.

21. Module 4 — Performance Prediction

Objective

Predict current race performance based on historical training.

Initial targets:

- 5K
- 10K
- Half Marathon

Example:

```
Predicted 10K

52:18
```

Expected Range

51:30 – 54:10

Confidence

Medium

Prediction should include uncertainty rather than presenting a false sense of precision.

22. Prediction Features

Potential features:

Recent 7-day mileage
Recent 28-day mileage
Recent average pace
Recent HR
Long-run distance
Workout frequency
Workout intensity
Elevation
Cadence
Training load
Days since last hard session
Recent race performance

Environmental variables can be included when appropriate.

23. Model Strategy

Because the current dataset contains approximately 150 runs, avoid unnecessary deep learning.

Start with strong classical models:

Baselines

- Naive baseline
- Historical best
- Moving average

- Linear regression

ML models

- ElasticNet
- Random Forest
- XGBoost / LightGBM

Compare models rather than assuming the most complex model is best.

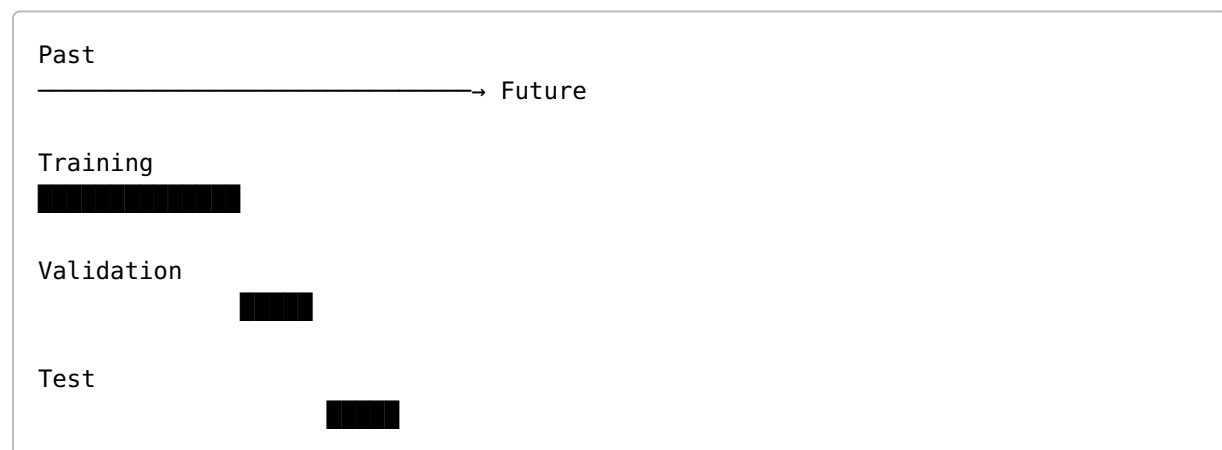
24. Time-Series Validation

This is a critical ML requirement.

Do NOT use:

Random Train/Test Split

Instead:



Potential approaches:

- Time-based train/validation/test split
- Expanding-window validation
- Rolling-window evaluation

The project should explicitly document leakage prevention.

25. Explainable ML

Predictions should answer:

Why did the model predict this?

Example:

Prediction Drivers

Recent Training Volume	↑
Recent Performance	↑
Consistency	↑
Fatigue	↓
Elevation	↓
Environmental Stress	↓

SHAP can be used for model explainability where appropriate.

26. Module 5 — Goals

The user defines a race goal.

Example:

Goal

10K < 50:00

Race Date

12 weeks

Current Prediction

52:18

Gap

2:18

The system tracks progress over time.

27. Goal Progress

Track:

- Current prediction
- Target performance
- Performance gap
- Trend
- Training consistency
- Weekly volume
- Key workout indicators

Example:

Week 1	54:02
Week 4	53:12
Week 8	52:18
Week 12	?

28. Future Training Plan

Later, generate a personalized training recommendation.

Example:

Current
10K prediction: 52:18
Target
10K: 49:59
Gap
2:19

The system identifies potential areas:

- Aerobic volume
- Long-run development
- Threshold work
- Speed work
- Consistency
- Recovery

The system should explain recommendations rather than simply output a generic training plan.

29. Module 6 — AI Running Coach

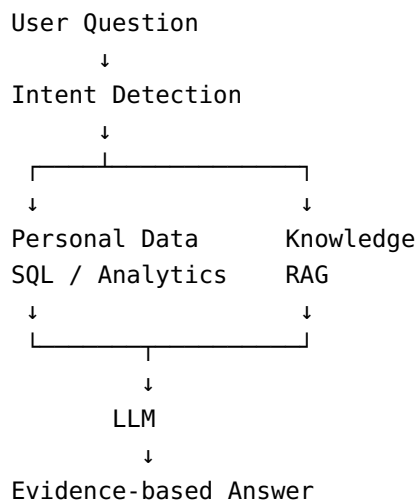
This is the main LLM feature.

The chatbot should be more than a generic ChatGPT wrapper.

It should combine:

```
Personal Data
+
Analytics
+
SQL
+
RAG Knowledge
+
LLM
```

30. AI Coach Architecture



31. AI Coach Capabilities

Example questions:

Performance

Why have I been running slower recently?

Weather

Does hot weather affect my pace?

Historical

What was my best month this year?

Comparison

How did today's run compare with similar runs?

Goal

Am I on track for sub-50 10K?

Training

How has my training volume changed?

Environment

Do I perform worse when humidity is high?

Exploration

What are the biggest factors associated with my performance?

32. SQL + RAG Architecture

Not every question should use vector search.

Structured questions

Use SQL:

How many km did I run last month?

SQL → Result → LLM

Analytical questions

Use analytics:

Am I running faster than six months ago?

Analytics → Result → LLM

Knowledge questions

Use RAG:

Why can heat affect running performance?

RAG → Relevant knowledge → LLM

Mixed questions

Use both:

Why was today's run slower than normal?

SQL
+
Weather
+
Historical comparison
+
RAG if necessary
↓
LLM

This should be a key architectural principle of the project.

33. Evidence-based Answers

The chatbot should provide supporting evidence.

Example:

Conclusion

Your recent slowdown appears to be associated with increased training load and warmer conditions.

Evidence

- Recent pace: -8 sec/km
- Training volume: +18%
- Average temperature: +3°C
- Similar-condition runs: 7

The AI should not fabricate evidence.

34. Module 7 — Running Form Analysis

Status

Advanced / Phase 3.

This should NOT block the core project.

35. Video Pipeline

```
Running Video
  ↓
Frame Extraction
  ↓
Pose Estimation
  ↓
Body Keypoints
  ↓
Temporal Analysis
  ↓
```

Running Form Metrics



Report

Potential technologies:

- MediaPipe
- YOLO Pose
- OpenPose
- OpenCV

36. Potential Metrics

Depending on video quality and camera angle:

- Cadence estimation
- Stride timing
- Knee angle
- Hip angle
- Trunk angle
- Left/right symmetry
- Vertical movement
- Joint movement patterns

The system can compare:

January Video
vs.
September Video

to identify changes.

37. Important Safety / Scientific Limitation

Do not initially market the system as:

Injury prediction

or:

Injury diagnosis

Instead:

Running Form Analysis

and:

Movement Pattern Insights

The system should clearly distinguish measurable observations from medical conclusions.

38. Recommended Tech Stack

Backend

Python

Primary language.

Data Processing

- Pandas
 - Polars if performance becomes relevant
 - NumPy
-

Data Validation

- Pandera
-

Database

PostgreSQL

Stores:

- Activities
- Metrics
- Weather
- AQI
- Training load

- Predictions
 - Goals
 - Insights
-

ML

Start with:

- scikit-learn
- XGBoost

Optional:

- LightGBM
 - SHAP
-

Experiment Tracking

MLflow

Track:

- Model version
 - Parameters
 - Metrics
 - Training dataset
 - Artifacts
-

Backend API

FastAPI

Responsibilities:

```
/api/activities  
/api/analytics  
/api/weather  
/api/predictions  
/api/goals  
/api/chat
```

Frontend

Recommended:

React / Next.js

or, if prioritizing speed of development:

Streamlit

For a strong portfolio project, React/Next.js gives a stronger product feel.

Visualization

Possible:

- Recharts
 - Plotly
 - ECharts
-

RAG

Possible components:

- Embedding model
- Vector database
- PostgreSQL + pgvector

Since this is a single-user system, PostgreSQL + pgvector is attractive because it avoids introducing another database unnecessarily.

LLM

Use an LLM API through a clean abstraction layer.

The application should not tightly couple business logic to one specific LLM provider.

Computer Vision

Phase 3:

- OpenCV
 - MediaPipe / YOLO Pose
 - Python
-

Infrastructure

- Docker
 - Docker Compose
 - GitHub Actions
 - Cloud deployment
-

Testing

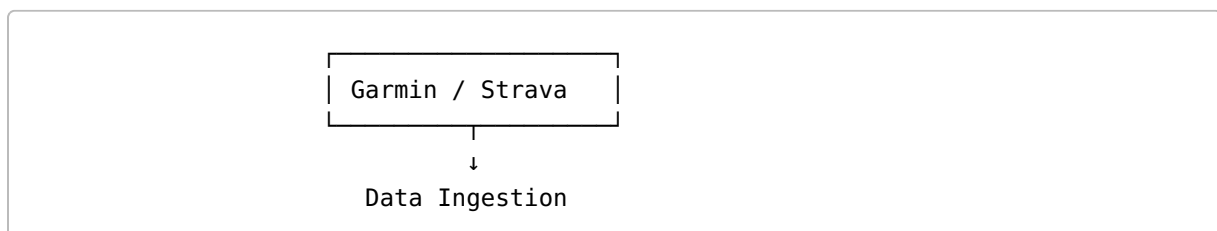
- pytest
 - API tests
 - Data pipeline tests
 - ML tests
 - RAG evaluation
-

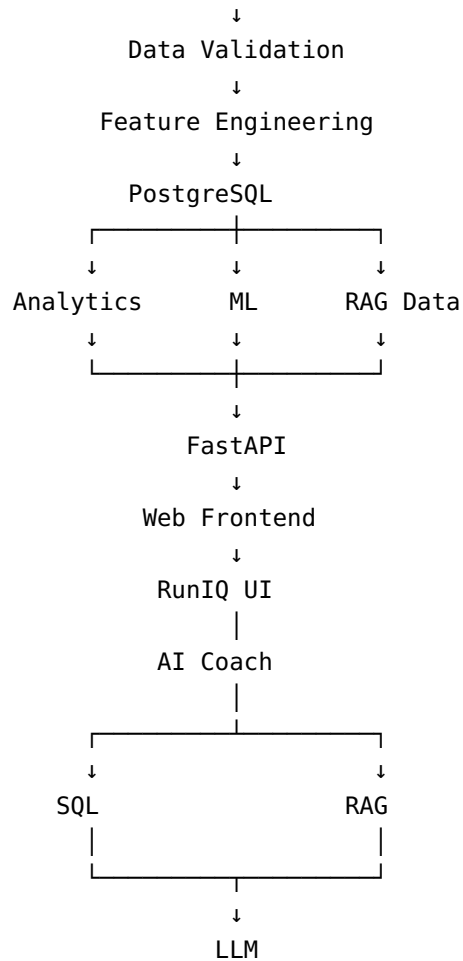
Monitoring

Potential:

- Evidently
 - Application logging
 - Model performance tracking
 - Data drift monitoring
-

39. Proposed System Architecture





40. Development Phases

Phase 0 — Project Definition

Duration: 2-3 days

Goals

Freeze:

- Product scope
- Architecture
- Tech stack
- Data sources
- MVP
- Non-goals

Deliverables

- PRD
- System architecture
- Repository structure
- Initial README
- Development roadmap

41. Phase 1 — Data Foundation

Duration: 1 week

Tasks

- Obtain Garmin / Strava export
- Inspect schema
- Build ingestion pipeline
- Clean data
- Validate data
- Normalize units
- Design PostgreSQL schema
- Load historical runs

Deliverable

```
Raw Data
  ↓
Validation
  ↓
Clean Data
  ↓
PostgreSQL
```

Definition of Done

You can import the entire historical dataset reproducibly with one pipeline.

42. Phase 2 — Weather & AQ Integration

Duration: 1 week

Tasks

- Integrate OpenWeather
- Integrate Google Air Quality API
- Match timestamps / locations
- Store environmental snapshots
- Handle missing API data
- Cache API responses
- Implement retry / error handling

Deliverable

Every eligible running activity has associated environmental information.

43. Phase 3 — Analytics Engine

Duration: 1–1.5 weeks

Tasks

Build:

- Basic statistics
- Weekly / monthly trends
- Personal baseline
- Training volume
- Training frequency
- Similar-run engine
- Weather impact analysis
- AQI analysis

Deliverable

A reusable analytics layer independent from the UI.

44. Phase 4 — ML Intelligence

Duration: 2 weeks

Tasks

Build:

1. Performance baseline
2. 5K prediction
3. 10K prediction
4. Half-marathon prediction if data supports it
5. Training-load indicators
6. Fitness / fatigue indicators
7. Weather-adjusted performance
8. Model explainability

Important

For every model document:

Problem
Target
Features
Baseline
Model
Validation
Metrics
Limitations

45. Phase 5 — Goal Intelligence

Duration: 1 week

Tasks

- Race goal creation
- Current performance estimate
- Target gap
- Progress tracking
- Trend estimation
- Goal status

Example:

10K Goal: <50:00

Current: 52:18

Status:
Improving

Gap:
2:18

46. Phase 6 — AI Running Coach

Duration: 1.5–2 weeks

Tasks

- Define chatbot architecture
- Build SQL tools
- Build analytics tools
- Build RAG pipeline
- Add personal data retrieval
- Add knowledge retrieval
- Implement response generation
- Add evidence / citations
- Evaluate hallucination
- Create conversation UI

Definition of Done

The AI can answer meaningful questions about the user's actual running history and provide evidence.

47. Phase 7 — Web Application

Duration: 1–1.5 weeks

Build:

Overview
Activities
Activity Detail
Analytics
Predictions
Goals

Focus on product usability rather than adding more features.

48. Phase 8 — Production Engineering

Duration: 1 week

Tasks

- Docker
 - Docker Compose
 - Environment variables
 - CI/CD
 - Unit tests
 - Integration tests
 - API tests
 - Database migrations
 - Logging
 - Error handling
 - Deployment
-

49. Phase 9 — Monitoring & Documentation

Duration: 3–5 days

Document:

- Architecture
- Data pipeline
- ML methodology
- Model evaluation
- RAG architecture
- API design
- Deployment
- Limitations
- Future work

Potential monitoring:

- API health
- Data quality

- Model drift
- Prediction performance
- RAG evaluation

50. Phase 10 — Advanced Computer Vision

Optional

Duration: 2–3 weeks

Only start after the core system is stable.

Build:

```
Video Upload
  ↓
Pose Estimation
  ↓
Movement Metrics
  ↓
Form Analysis
  ↓
Report
```

This becomes the project's advanced Computer Vision extension.

51. 12-Week Suggested Timeline

```
Week 1
Project setup
Data exploration
Database

Week 2
Data pipeline
Validation
Weather / AQ

Week 3
Analytics engine
```

Week 4

Personal baseline

Similar runs

Environmental analytics

Week 5

ML baselines

Performance prediction

Week 6

Training intelligence

Weather-adjusted performance

Explainability

Week 7

Goals

Race prediction

Progress tracking

Week 8

RAG architecture

SQL tools

Week 9

AI Coach

Evaluation

Week 10

Frontend

Dashboard

Week 11

Docker

Testing

CI/CD

Deployment

Week 12

Monitoring

Documentation

Polishing

Demo

Optional Week 13–14+

Running Form / Computer Vision

52. MVP Definition

The project is considered a successful MVP when the following works:

```
Garmin / Strava Data
  ↓
Data Pipeline
  ↓
PostgreSQL
  ↓
Weather + AQ
  ↓
Analytics
  ↓
ML Prediction
  ↓
Goal Tracking
  ↓
AI Coach
  ↓
Web Dashboard
```

The user should be able to answer:

How am I doing?

Am I improving?

What affects my performance?

How does weather affect me?

What is my current 10K prediction?

Am I on track for my goal?

Why was today's run different?

What does my historical data tell me?

53. What NOT to Build Initially

To prevent scope creep, do not initially build:

- Mobile app
- Social network
- Public user profiles
- Subscription system
- Multi-user permissions
- Real-time GPS tracking
- Generic workout marketplace
- Full medical injury prediction
- Complex deep-learning models
- Full computer vision system
- Generic AI chatbot

These can be future projects/features.

54. Portfolio Story

The project should eventually be presented as more than:

"I built a running dashboard."

The portfolio story should be:

I built a personalized running intelligence platform that integrates longitudinal activity data with environmental conditions to model individual performance, explain performance changes, forecast race outcomes, and provide evidence-based insights through an LLM-powered running coach.

55. Engineering Skills Demonstrated

This project can demonstrate:

Data Engineering

- ETL
- Data validation
- API integration
- Database design
- Data quality

Machine Learning

- Feature engineering
- Regression
- Time-series validation
- Model comparison
- Explainability
- Prediction uncertainty

LLM / AI

- RAG
- Tool calling
- SQL-based retrieval
- Personal knowledge retrieval
- AI evaluation
- Hallucination prevention

Backend

- FastAPI
- PostgreSQL
- REST API
- Authentication if later required

Frontend

- Dashboard
- Data visualization
- AI chat interface

DevOps

- Docker
- CI/CD
- Cloud deployment
- Monitoring

Advanced AI

- Computer vision
- Pose estimation
- Temporal analysis

56. Project Success Metrics

The project should have technical metrics, not just "website works."

Data

- Data ingestion success rate
- Validation error rate
- Missing environmental data rate

ML

- MAE
- RMSE
- R^2 where appropriate
- Prediction interval coverage
- Baseline comparison

RAG

- Retrieval relevance
- Answer groundedness
- Hallucination rate
- Correct SQL/tool usage

Product

Potential personal metrics:

- Prediction usefulness
- Insight usefulness
- Number of questions answered correctly
- User interaction frequency

Because this is a single-user project, qualitative evaluation is acceptable alongside technical metrics.

57. Key Technical Risks

Risk 1 — Too little data

Current dataset:

~150 runs.

Solution:

Use classical ML and strong baselines.

Avoid deep learning unless sufficient data becomes available.

Risk 2 — Data leakage

Especially important for performance prediction.

Solution:

Strict temporal validation.

Risk 3 — Weather correlation $\neq$ causation

Solution:

Control for:

- Fitness
- Distance
- Training load
- Workout type
- Elevation
- HR

Use careful language.

Risk 4 — LLM hallucination

Solution:

The AI Coach should retrieve evidence before making personalized claims.

Risk 5 — Scope creep

The biggest risk.

Solution:

Freeze MVP.

Video analysis is explicitly Phase 3.

58. Definition of "Done"

The project is not finished when:

"The website looks good."

It is finished when:

Data

Raw data can be reproducibly processed.

ML

Models have baselines, evaluation and documented limitations.

AI

The chatbot can answer questions using real personal data.

Engineering

The application is containerized, tested and deployable.

Product

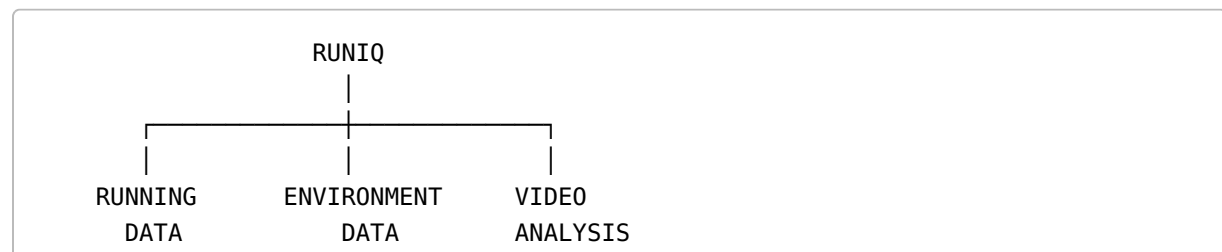
A runner can actually use the system to understand their training.

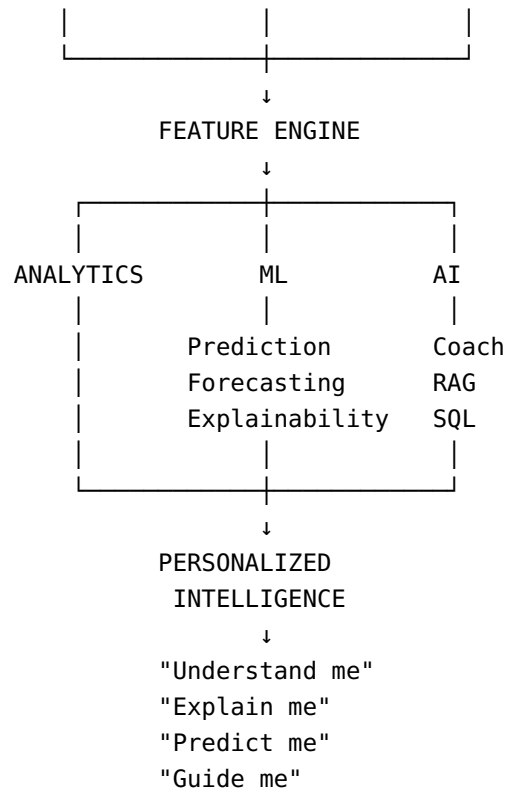
Documentation

Another developer can understand the architecture and reproduce the project.

59. Final Product Vision

The long-term RunIQ architecture is:





60. Guiding Principle

The most important principle for the entire project:

Do not add a feature just because it uses AI.

Every feature should answer a meaningful question for the runner.

Examples:

Dashboard

→ What is happening?

Analytics

→ What has happened?

Weather Intelligence

→ Why might it have happened?

Prediction

→ What might happen next?

Goals

→ Am I on track?

AI Coach

→ Help me understand and make decisions.

Video Analysis

→ How is my running movement changing?

If every module follows this principle, RunIQ will feel like one coherent product rather than a collection of unrelated AI demos.